Nevil C Prince
Email: ep23btech11019@iith.ac.in
Course: Multi-Agent Systems
Instructor: Venkatraman Renganathan

Multi-Agent Systems – Assignment 2
Roll No: EP23BTECH11019
Due Date: September 4, 2026

Problem 1: Graph Theory and Consensus Dynamics

Problem 1(a)
Prove that $v^\top x[k]$ is constant over all time steps $k \in \mathbb{N}$.

Solution: The consensus system evolves as $x[k+1] = Ax[k]$, where A is row-stochastic and v is its normalized left eigenvector, so $v^\top A = v^\top$ and $\mathbf{1}_N^\top v = 1$.

Look at what happens to $v^\top x[k]$ one step later:

$$\begin{aligned} v^\top x[k+1] &= v^\top (Ax[k]) \\ &= (v^\top A)\, x[k] && \text{(matrix multiplication is associative)} \\ &= v^\top x[k] && \text{(using } v^\top A = v^\top\text{)} \end{aligned}$$

So $v^\top x[k+1] = v^\top x[k]$, and this holds for any k. Applying it repeatedly gives

$$v^\top x[k] = v^\top x[k-1] = \cdots = v^\top x[0], \qquad \forall k \in \mathbb{N},$$

which means $v^\top x[k]$ never changes – it stays fixed at its initial value for all time. □

Problem 1(b)
Show that $(A - \mathbf{1}_N v^\top) = (A - \mathbf{1}_N v^\top)(I_N - \mathbf{1}_N v^\top)$.

Solution: Start from the right-hand side and multiply it out:

$$\begin{aligned} (A - \mathbf{1}_N v^\top)(I_N - \mathbf{1}_N v^\top) &= AI_N - A\mathbf{1}_N v^\top - \mathbf{1}_N v^\top I_N + \mathbf{1}_N v^\top \mathbf{1}_N v^\top \\ &= A - A\mathbf{1}_N v^\top - \mathbf{1}_N v^\top + \mathbf{1}_N (v^\top \mathbf{1}_N) v^\top \end{aligned}$$

Two facts simplify this further:

(i) A is row-stochastic, so every row sums to 1: $A\mathbf{1}_N = \mathbf{1}_N$.

(ii) v satisfies $\mathbf{1}_N^\top v = 1$, and since this is just a scalar, $v^\top \mathbf{1}_N = 1$ too.

Plugging these in:

$$\begin{aligned} (A - \mathbf{1}_N v^\top)(I_N - \mathbf{1}_N v^\top) &= A - \mathbf{1}_N v^\top - \mathbf{1}_N v^\top + \mathbf{1}_N (1) v^\top \\ &= A - \mathbf{1}_N v^\top - \mathbf{1}_N v^\top + \mathbf{1}_N v^\top \\ &= A - \mathbf{1}_N v^\top. \end{aligned}$$

That's exactly the left-hand side, so the identity holds. □

Problem 1(c)
Show that the dynamics of the disagreement vector are given by $\delta[k+1] = (A - \mathbf{1}_N v^\top)\, \delta[k]$.

Solution: By definition, $\delta[k] = x[k] - (v^\top x[k])\mathbf{1}_N$, so at the next time step

$$\delta[k+1] = x[k+1] - (v^\top x[k+1])\mathbf{1}_N.$$

Substitute $x[k+1] = Ax[k]$, and use the fact from part (a) that $v^\top x[k+1] = v^\top x[k]$:

$$\delta[k+1] = Ax[k] - (v^\top x[k])\mathbf{1}_N = Ax[k] - \mathbf{1}_N v^\top x[k] = (A - \mathbf{1}_N v^\top)\, x[k],$$

where the middle step just reorders the scalar $v^\top x[k]$ next to $\mathbf{1}_N$.

What's left is to check that $(A - \mathbf{1}_N v^\top)x[k]$ equals $(A - \mathbf{1}_N v^\top)\delta[k]$. Expanding the latter:

$$\begin{aligned}(A - \mathbf{1}_N v^\top)\,\delta[k] &= (A - \mathbf{1}_N v^\top)\big(x[k] - \mathbf{1}_N v^\top x[k]\big) \\ &= (A - \mathbf{1}_N v^\top)(I_N - \mathbf{1}_N v^\top)\, x[k].\end{aligned}$$

Part (b) already showed $(A - \mathbf{1}_N v^\top)(I_N - \mathbf{1}_N v^\top) = A - \mathbf{1}_N v^\top$, so this reduces to

$$(A - \mathbf{1}_N v^\top)\,\delta[k] = (A - \mathbf{1}_N v^\top)\, x[k].$$

Putting this back into the expression for $\delta[k+1]$ gives the result:

$$\boxed{\delta[k+1] = (A - \mathbf{1}_N v^\top)\,\delta[k].}$$

□

Problem 2: Resilient Consensus, Robustness, and Privacy

Problem 2.1
Consider the given 3-robust graph, which offers robustness against $F = 1$ local adversaries. Simulate the resilient consensus of this graph topology using the W-MSR protocol, with random initial conditions and an adversary free to behave arbitrarily.

Solution: **Setup.** I index the nodes 0–7, matching nodes 1–8 in the figure, and build the adjacency list directly from the diagram. Node 8 (index 7) plays the role of the local adversary – each benign node is allowed at most one such "bad friend," which is what the $F = 1$ robustness guarantee is meant for. To make the effect of the attack obvious, I set the adversary to a fixed extreme value ($x_{\text{adv}} = 100$) instead of a mild perturbation, while every benign node starts from a random initial condition.

W-MSR update rule. At every iteration, each node looks at its own value together with its neighbors' values, sorts them, and throws out up to F values that are strictly larger and up to F that are strictly smaller than its own current value (as many as are available). It then sets its new value to the average of what remains, including its own value.

Inference. Since the adversary always transmits the same extreme value, it keeps landing among the largest values in every benign neighbor's neighborhood, so with $F = 1$ it gets trimmed off before the averaging step, iteration after iteration. Because each benign node has at most one adversarial neighbor – that's exactly what 3-robustness with $F = 1$ guarantees – this filtering shuts the adversary out completely, and the benign nodes settle into a shared consensus value that stays within the range of their initial states, unaffected by the injected outlier.

Listing 1: W-MSR resilient consensus simulation

```
import numpy as np
import matplotlib.pyplot as plt

# Adjacency list for the 3-robust graph (0-indexed; nodes 1-8 in the figure -> 0-7 here)
adj = {
    0: [1, 2, 4, 6],        # Node 1
    1: [0, 3, 4, 5],        # Node 2
    2: [0, 3, 4, 6],        # Node 3
    3: [1, 2, 5, 7],        # Node 4
    4: [5, 6, 7, 0, 1, 2], # Node 5
    5: [4, 6, 7, 1, 3],     # Node 6
    6: [4, 5, 7, 0, 2],     # Node 7
    7: [4, 5, 6, 3]         # Node 8
```

```
}

N = 8
F = 1          # robust against F = 1 local adversary
iters = 20
adversary = 7  # Node 8 (index 7) is the malicious agent

np.random.seed(42)
x = np.random.uniform(0, 10, N)
x[adversary] = 100.0  # adversary starts (and stays) at an extreme value

history = [x.copy()]

for k in range(iters):
    x_next = np.zeros(N)
    for i in range(N):
        if i == adversary:
            x_next[i] = 100.0  # adversary maintains its malicious extreme value
            continue

        neighbor_vals = [x[j] for j in adj[i]]
        neighbor_vals.sort()

        # W-MSR filtering: remove up to F strictly larger and F strictly smaller
        larger = [v for v in neighbor_vals if v > x[i]]
        smaller = [v for v in neighbor_vals if v < x[i]]
        removed_larger = larger[-F:] if larger else []
        removed_smaller = smaller[:F] if smaller else []

        M_i = []
        for v in neighbor_vals:
            if v in removed_larger:
                removed_larger.remove(v)
            elif v in removed_smaller:
                removed_smaller.remove(v)
            else:
                M_i.append(v)

        M_i.append(x[i])  # keep own value
        x_next[i] = sum(M_i) / len(M_i)

    x = x_next.copy()
    history.append(x.copy())

history = np.array(history)

plt.figure(figsize=(8, 5))
for i in range(N):
    if i == adversary:
        plt.plot(history[:, i], 'r--', linewidth=2, label=f'Adversary (Node {i+1})')
    else:
        plt.plot(history[:, i], label=f'Benign Node {i+1}')

plt.title('W-MSR Resilient Consensus (F = 1)')
plt.xlabel('Time Step k')
plt.ylabel('State Value x_i[k]')
plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left')
plt.grid(True)
plt.tight_layout()
plt.savefig('wmsr_plot.png', dpi=150)
plt.show()
```

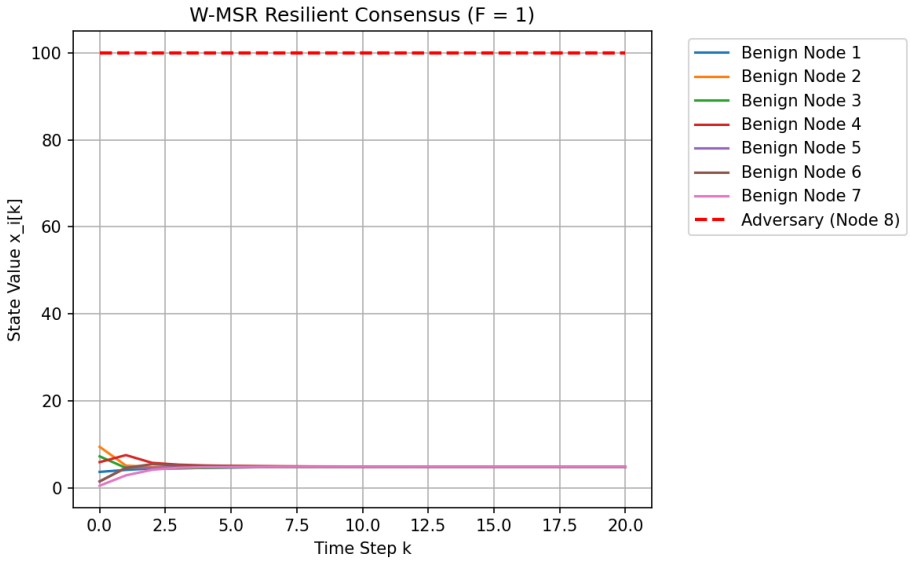


Figure 1: State trajectories $x_i[k]$ under the W-MSR protocol. Benign nodes converge to a common value while the adversary (dashed red) remains fixed at its extreme value and is filtered out.

Problem 2.2
Argue, by constructing/drawing subsets, why this 3-robust graph is *not* a $(3, 2)$-robust graph.

Solution: By definition, $\mathcal{G}$ is $(3, 2)$-robust only if *every* pair of disjoint nonempty subsets $S_1, S_2 \subset \mathcal{V}$ satisfies at least one of:

(i) $|X^3_{S_1}| = |S_1|$,

(ii) $|X^3_{S_2}| = |S_2|$,

(iii) $|X^3_{S_1}| + |X^3_{S_2}| \geq 2$,

where X^3_S is the set of nodes in S that have 3 or more neighbors outside S. So to show the graph is *not* $(3, 2)$-robust, I just need one partition where all three of these fail at once.

Partition used. Take $S_1 = \{1, 2, 3, 4\}$ (the outer four nodes) and $S_2 = \{5, 6, 7, 8\}$ (the inner four), following the layout in the figure.

Checking S_1: each outer node has exactly two neighbors in S_2:

- Node $1 \to \{5, 7\}$: 2 outside neighbors, so $1 \notin X^3_{S_1}$.
- Node $2 \to \{5, 6\}$: 2 outside neighbors, so $2 \notin X^3_{S_1}$.
- Node $3 \to \{5, 7\}$: 2 outside neighbors, so $3 \notin X^3_{S_1}$.
- Node $4 \to \{6, 8\}$: 2 outside neighbors, so $4 \notin X^3_{S_1}$.

So $|X^3_{S_1}| = 0$, which is not equal to $|S_1| = 4$ – condition (i) fails.

Checking S_2:

- Node $5 \to \{1, 2, 3\}$: 3 outside neighbors, so $5 \in X^3_{S_2}$.
- Node $6 \to \{2, 4\}$: 2 outside neighbors, so $6 \notin X^3_{S_2}$.
- Node $7 \to \{1, 3\}$: 2 outside neighbors, so $7 \notin X^3_{S_2}$.
- Node $8 \to \{4\}$: 1 outside neighbor, so $8 \notin X^3_{S_2}$.

So $|X^3_{S_2}| = 1$, not equal to $|S_2| = 4$ – condition (ii) fails too.

Sum: $|X^3_{S_1}| + |X^3_{S_2}| = 0 + 1 = 1$, which is less than 2 – so condition (iii) fails as well.

All three conditions fail for this single partition, which is enough to conclude the graph is **not** $(3, 2)$-robust, even though it is 3-robust. □

Problem 2.3
Suppose a "good" eavesdropper monitors all communication links, identifies the malicious agent, and secretly modifies its transmitted values to protect benign neighbors. What strategy could the malicious agent use to evade the eavesdropper?

Solution: If the eavesdropper is actively catching and correcting obviously bad transmissions – like a node broadcasting a fixed value of 100 – then the adversary's old trick of just picking an extreme constant is no longer viable. That kind of glaring deviation is exactly what a good eavesdropper is built to catch.

So the adversary has to get subtler. Instead of transmitting something wildly off, it can watch what its neighbors are sending (it has access to the same links, after all) and nudge its own reported value just slightly away from theirs – small enough that it neither looks anomalous to the eavesdropper nor gets pushed into the top/bottom F values that W-MSR's sorting step would trim. Individually, each of these small deviations is invisible to both defenses.

The catch is that "small per step" doesn't mean "small overall." If the adversary keeps injecting this tiny bias consistently over many rounds rather than just once, the drift accumulates. W-MSR's guarantees are built around catching outliers, not around bounding the effect of a persistent, low-magnitude skew, so over time this can still nudge the whole network's consensus value away from where it should have converged.

Basically, the adversary is trading size for stealth: give up the big, obvious attack in exchange for a slow one that neither the eavesdropper nor W-MSR is designed to notice. It's a useful reminder that being robust against sudden extreme values isn't the same as being robust against a patient, low-and-slow attack.

Problem 2.4
Should benign agents run W-MSR directly on encrypted neighbor values, or decrypt first and then run W-MSR? Justify your choice, note the subsequent problem this creates, and summarize two seminal papers addressing it (one paragraph each, no cryptographic-algorithm details).

Solution: **My answer:** decrypt first, then run W-MSR.

Why. W-MSR only works because it can sort the incoming values and cut off the top and bottom F. That sorting step needs numbers that are comparable in a meaningful sense. Ordinary encryption schemes don't preserve that kind of ordering – ciphertext A being "larger" than ciphertext B says nothing about the underlying plaintexts. So if a node tried to sort encrypted neighbor values directly, the resulting order would be essentially random relative to the true states. That means the filtering step could just as easily throw away a legitimate benign value and keep the adversary's, which defeats the entire point of running W-MSR in the first place.

But this creates a new problem. Decrypting before filtering fixes the ordering issue, but it means each node now sees its neighbors' actual plaintext states, which is exactly what the encryption was supposed to prevent. If a benign node is later compromised, or is simply "honest-but-curious," it now has direct access to its neighbors' private values and can leak them. So encryption alone doesn't really buy you privacy once W-MSR needs to run on the decrypted data – you need something that lets the filtering happen without any single node ever seeing everyone else's raw state, which is why this is an active research area combining resilience and privacy rather than treating them separately.

Two papers on this:

Mo and Murray, "Privacy Preserving Average Consensus," IEEE Transactions on Automatic Control, 2017. Instead of encryption, this paper's approach is to have each agent add carefully designed random noise to its transmitted state – noise that sums to zero across the network and decays to zero over time. Because of that structure, the network still converges to the exact true average in the long run, but at any given moment, a neighbor (or an eavesdropper) only sees a noisy version of the state, not the real one. The initial condition ends up protected in a differential-privacy sense, without needing cryptography at all.

Ruan, Gao, and Wang, "Secure and Privacy-Preserving Consensus," IEEE Transactions on Automatic Control, 2019. This paper handles a tougher setting – privacy has to hold even if some neighbors collude or an eavesdropper is watching every link, which can eventually break simple noise-based schemes if given enough observations to average out over time. Their fix is to have each node split its true value into multiple correlated pieces and hand those pieces out to different neighbors, so that the necessary computation for consensus can still happen collectively using just those pieces, without any single node ever reconstructing another node's actual state.